Aplicación CRUD de Gestión de Libros con HTML, CSS y JavaScript

David Raúl Sánchez Pincay

1. Objetivo

Desarrollar una aplicación web del lado del cliente (front-end) que permita gestionar un listado de libros (Título y Autor) mediante las cuatro operaciones básicas de un CRUD: **Crear**, **Leer**, **Actualizar** y **Eliminar**. La aplicación utiliza HTML para la estructura, CSS para la presentación y JavaScript puro (sin librerías) para la lógica, almacenando los datos en el navegador mediante `localStorage`, por lo que no requiere servidor ni base de datos.

2. Estructura del proyecto

```
CRUD/
 |-- index.html      (estructura y contenido)
 |-- estilos.css     (hoja de estilos externa)
 `-- aplicacion.js   (lógica del CRUD y almacenamiento local)
```

3. Repositorio Git

https://github.com/matsuro262/aplicaciones_cliente_web

El proyecto se gestiona con control de versiones Git y está listo para subirse al repositorio remoto indicado.

4. Explicación del código HTML (index.html)

Todo el contenido está dentro de un `<div class="contenedor">` que centra y limita el ancho de la página. El documento se divide en un encabezado y un área principal con dos secciones:

- **<header>**: título “Gestión de Libros” y subtítulo del taller.
- **.seccion-formulario**: formulario `#formulario-crud` con un campo oculto `#id-elemento` (guarda el ID del libro en edición), los campos de texto `#titulo` y `#autor` (ambos `required`) y dos botones: `#boton-guardar` (tipo submit) y `#boton-cancelar` (oculto por defecto con la clase `.oculto`).
- **.seccion-tabla**: tabla con encabezados Título, Autor y Acciones; el cuerpo `<tbody id="cuerpo-tabla">` se llena dinámicamente desde JavaScript.
- Al final del `<body>` se enlaza `aplicacion.js`, de modo que el DOM ya existe cuando el script se ejecuta.
- La etiqueta `<meta name="viewport">` permite que la página se adapte a pantallas móviles.

5. Código HTML

```html
<!doctype html>
<html lang="es">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Taller CRUD - Cliente Web</title>
    <link rel="stylesheet" href="estilos.css" />
</head>
```

```
<body>
    <div class="contenedor">
        <header>
            <h1>Gestión de Libros</h1>
            <p>Taller práctico de aplicaciones web front-end</p>
        </header>
        <main>
            <section class="seccion-formulario">
                <h2>Añadir / Editar Libro</h2>
                <form id="formulario-crud">
                    <input type="hidden" id="id-elemento" />

                    <div class="grupo-formulario">
                        <label for="titulo">Título del Libro:</label>
                        <input type="text" id="titulo" required placeholder="Ej. Cien
años de
soledad" />
                    </div>
                    <div class="grupo-formulario">
                        <label for="autor">Autor:</label>
                        <input type="text" id="autor" required placeholder="Ej. Gabriel
García
Márquez" />
                    </div>
                    <button type="submit" id="boton-guardar">Guardar Libro</button>
                    <button type="button" id="boton-cancelar" class="oculto">
                        Cancelar Edición
                    </button>
                </form>
            </section>
            <section class="seccion-tabla">
                <h2>Listado de Libros</h2>
                <div class="tabla-responsiva">
                    <table>
                        <thead>
                            <tr>
                                <th>Título</th>
                                <th>Autor</th>
                                <th>Acciones</th>
                            </tr>
                        </thead>
                        <tbody id="cuerpo-tabla"></tbody>
                    </table>
                </div>
            </section>
        </main>
    </div>
```

```
    <script src="aplicacion.js"></script>
</body>

</html>
```

6. Explicación de la hoja de estilos (estilos.css)

- Se enlaza con `<link rel="stylesheet" href="estilos.css">`, separando la presentación del contenido.
- **Variables CSS** en `:root` (`--color-primario`, `--color-secundario`, `--color-exito`, `--color-peligro`, `--fondo-claro`) para mantener una paleta de colores consistente y fácil de cambiar.
- **Diseño con CSS Grid**: `main` usa una sola columna en pantallas pequeñas y, con la media query `@media (min-width: 768px)`, pasa a dos columnas (`1fr 2fr`): formulario a la izquierda y tabla a la derecha.
- Tarjetas blancas (`.seccion-formulario`, `.seccion-tabla`) con `border-radius` y `box-shadow`.
- Botones con colores semánticos: verde para guardar, azul para editar, rojo para eliminar y gris para cancelar; efecto `:hover` con `opacity`.
- Clase utilitaria `.oculto { display: none; }` que JavaScript agrega o quita para mostrar el botón de cancelar.
- Tabla con `border-collapse`, encabezado oscuro y resaltado de fila al pasar el cursor (`tr:hover`).

7. Código CSS

```
:root {
    --color-primario: #2c3e50;
    --color-secundario: #3498db;
    --color-exito: #2ecc71;
    --color-peligro: #e74c3c;
    --fondo-claro: #f8f9fa;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: var(--fondo-claro);
    margin: 0;
    padding: 20px;
    color: #333;
}

.contenedor {
    max-width: 900px;
    margin: 0 auto;
}

header {
    text-align: center;
```

```
    margin-bottom: 30px;
}

main {
    display: grid;
    grid-template-columns: 1fr;
    gap: 30px;
}

@media (min-width: 768px) {
    main {
        grid-template-columns: 1fr 2fr;
    }
}

/* Estilos del Formulario */
.seccion-formulario,
.seccion-tabla {
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.grupo-formulario {
    margin-bottom: 15px;
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

input[type="text"] {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 4px;
}

button {
    padding: 10px 15px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
```

```
    color: white;
    background-color: var(--color-secundario);
}

button:hover {
    opacity: 0.9;
}

#boton-guardar {
    background-color: var(--color-exito);
    width: 100%;
}

.btn-editar {
    background-color: var(--color-secundario);
    margin-right: 5px;
}

.btn-eliminar {
    background-color: var(--color-peligro);
}

#boton-cancelar {
    background-color: #7f8c8d;
    width: 100%;
    margin-top: 5px;
}

.oculto {
    display: none;
}

/* Estilos de la Tabla */
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
}

th,
td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}

th {
    background-color: var(--color-primario);
```

```
    color: white;
}

tr:hover {
    background-color: #f1f1f1;
}
```

8. Explicación del código JavaScript (aplicacion.js)

El archivo está organizado en cuatro bloques:

- **1. Estado de la aplicación**: el arreglo `libros` se carga desde `localStorage` (clave `listaLibros`) con `JSON.parse`, o se inicia vacío. La variable `estaEditando` indica si el formulario está en modo edición.
- **2. Elementos del DOM**: se capturan con `document.getElementById` el formulario, los campos, el cuerpo de la tabla y los botones.
- **3. Funciones del CRUD** (detalladas en la tabla siguiente).
- **4. Inicialización**: al final se llama a `renderizarLibros()` para dibujar los datos guardados al abrir la página.

Operación	Función / evento	Descripción
Crear	`submit` del formulario (rama `else`)	Evita la recarga con `preventDefault()`, toma los valores con `trim()`, crea un objeto `{id, titulo, autor}` con ID único generado por `crypto.randomUUID()` y lo agrega con `push`.
Leer	`renderizarLibros()`	Vacía el `tbody`, muestra "No hay libros registrados." si el arreglo está vacío; en caso contrario crea una fila `<tr>` por libro con sus botones Editar y Eliminar, y guarda el arreglo en `localStorage`.
Actualizar	`prepararEdicion(id)` y `submit` (rama `if`)	Primero busca el libro con `find`, carga sus datos en el formulario, cambia el botón a "Actualizar Libro" y muestra "Cancelar Edición". Al enviar, `map` reemplaza solo el libro cuyo ID coincide.
Eliminar	`eliminarLibro(id)`	Pide confirmación con `confirm()` y, si se acepta, elimina el registro con `filter`. Si el libro eliminado era el que se estaba editando, limpia el formulario.
Cancelar	`reiniciarFormulario()`	Limpia los campos, borra el ID oculto, desactiva el modo edición y oculta el botón de cancelar.

9. Código JavaScript

```
// --- 1. ESTADO DE LA APLICACIÓN ---
let libros = JSON.parse(localStorage.getItem('listaLibros')) || [];
let estaEditando = false;
// --- 2. ELEMENTOS DEL DOM (CAPTURA DE CAMPOS) ---
const formulario = document.getElementById('formulario-crud');
const entradaTitulo = document.getElementById('titulo');
const entradaAutor = document.getElementById('autor');
const entradaId = document.getElementById('id-elemento');
const cuerpoTabla = document.getElementById('cuerpo-tabla');
```

```
const botonGuardar = document.getElementById('boton-guardar');
const botonCancelar = document.getElementById('boton-cancelar');
// --- 3. FUNCIONES OPERATIVAS (CRUD) ---
// [LEER] Mostrar los datos almacenados en la tabla HTML
function renderizarLibros() {
    cuerpoTabla.innerHTML = ''; // Vaciar la tabla antes de volver a dibujar

    if (libros.length === 0) {
        cuerpoTabla.innerHTML = `<tr><td colspan="3" style="text-align:center;">No hay
libros registrados.</td></tr>`;
        return;
    }
    libros.forEach(libro => {
        const fila = document.createElement('tr');
        fila.innerHTML = `
 <td>${libro.titulo}</td>
 <td>${libro.autor}</td>
 <td>
 <button class="btn-editar"
onclick="prepararEdicion('${libro.id}')">Editar</button>
 <button class="btn-eliminar"
onclick="eliminarLibro('${libro.id}')">Eliminar</button>
 </td>
 `;
        cuerpoTabla.appendChild(fila);
    });

    // Guardar el estado actual en el almacenamiento local del navegador
    localStorage.setItem('listaLibros', JSON.stringify(libros));
}
// [CREAR Y ACTUALIZAR] Escuchar el envío del formulario
formulario.addEventListener('submit', (evento) => {
    evento.preventDefault(); // Detener la recarga por defecto de la página
    const valorTitulo = entradaTitulo.value.trim();
    const valorAutor = entradaAutor.value.trim();
    const idActual = entradaId.value;
    if (estaEditando) {
        // [ACTUALIZAR] Mapear el arreglo y modificar el registro que coincida con el
ID
        libros = libros.map(libro =>
            libro.id === idActual ? { ...libro, titulo: valorTitulo, autor: valorAutor
} : libro
        );
        estaEditando = false;
        botonGuardar.textContent = 'Guardar Libro';
        botonCancelar.classList.add('oculto');
    } else {
        // [CREAR] Construir un nuevo objeto libro con un ID único robusto
```

```
        const nuevoLibro = {
            id: crypto.randomUUID(),
            titulo: valorTitulo,
            autor: valorAutor
        };
        libros.push(nuevoLibro);
    }
    reiniciarFormulario();
    renderizarLibros();
});
// [ACTUALIZAR - PREPARACIÓN] Cargar el registro seleccionado al formulario
window.prepararEdicion = function (id) {
    const libroEncontrado = libros.find(libro => libro.id === id);
    if (!libroEncontrado) return;
    entradaTitulo.value = libroEncontrado.titulo;
    entradaAutor.value = libroEncontrado.autor;
    entradaId.value = libroEncontrado.id;
    estaEditando = true;
    botonGuardar.textContent = 'Actualizar Libro';
    botonCancelar.classList.remove('oculto');
};
// [ELIMINAR] Quitar un registro del arreglo
window.eliminarLibro = function (id) {
    if (confirm('¿Está seguro de que desea eliminar este libro?')) {
        libros = libros.filter(libro => libro.id !== id);

        // Si borramos el libro que se estaba editando en ese instante, limpiamos el
        formulario
        if (estaEditando && entradaId.value === id) {
            reiniciarFormulario();
        }

        renderizarLibros();
    }
};
// Cancelar el estado de edición de forma explícita
botonCancelar.addEventListener('click', reiniciarFormulario);
function reiniciarFormulario() {
    formulario.reset();
    entradaId.value = '';
    estaEditando = false;
    botonGuardar.textContent = 'Guardar Libro';
    botonCancelar.classList.add('oculto');
}
// --- 4. INICIALIZACIÓN DE LA APLICACIÓN ---
renderizarLibros();
```

10. Ejecución - Resultado en el navegador

A continuación se muestra el funcionamiento real de la aplicación, ejecutada en el navegador (Chromium) abriendo `index.html`. Se usaron libros de ejemplo para probar cada operación.

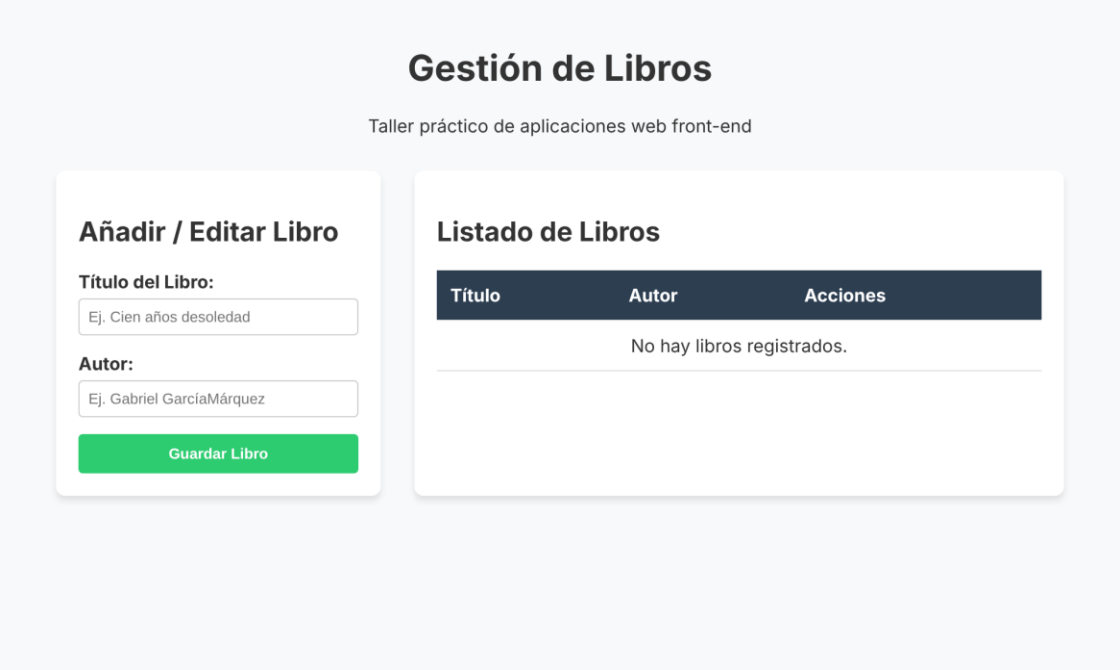


Figura 1. Estado inicial: sin libros registrados (se muestra el mensaje “No hay libros registrados.”).

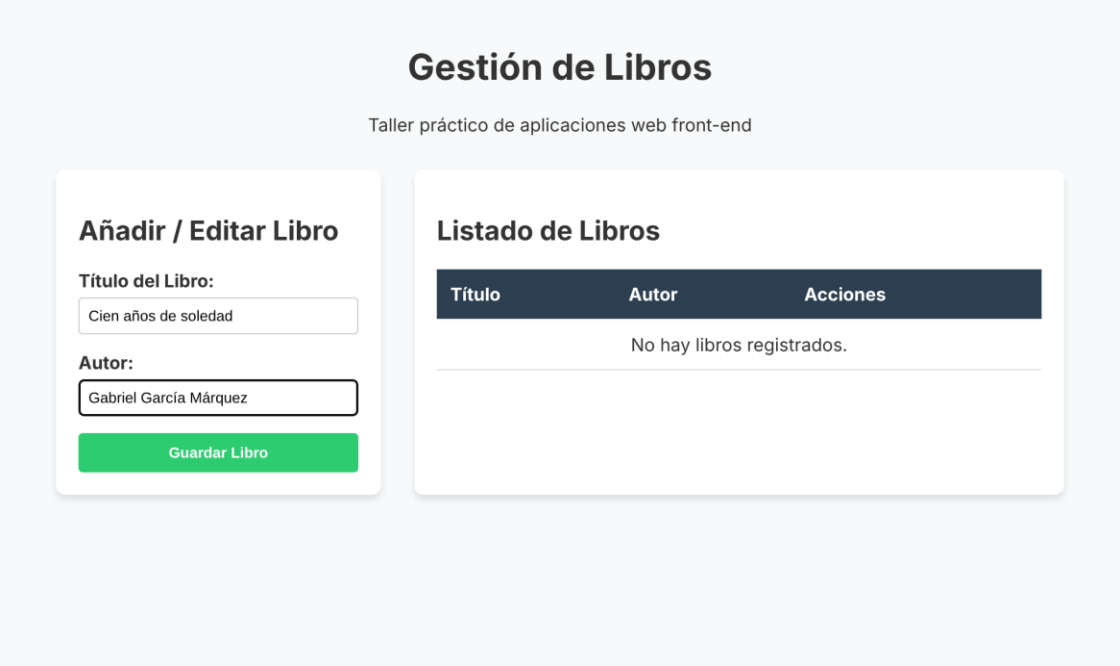


Figura 2. **Crear**: formulario completado antes de presionar “Guardar Libro”.

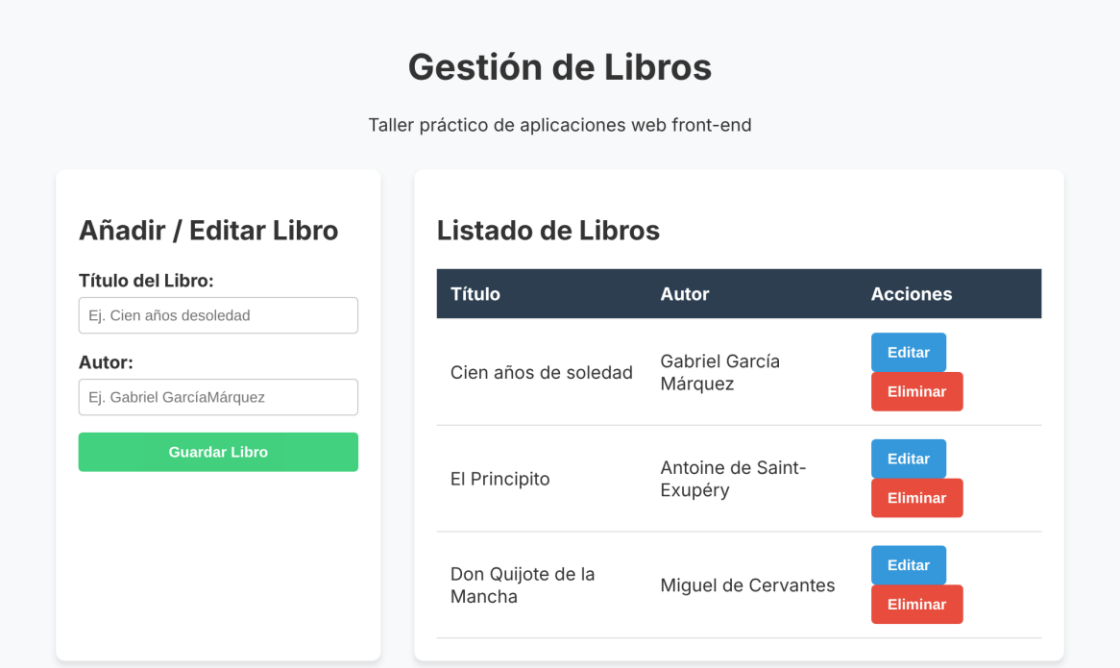


Figura 3. **Leer**: listado con tres libros registrados.

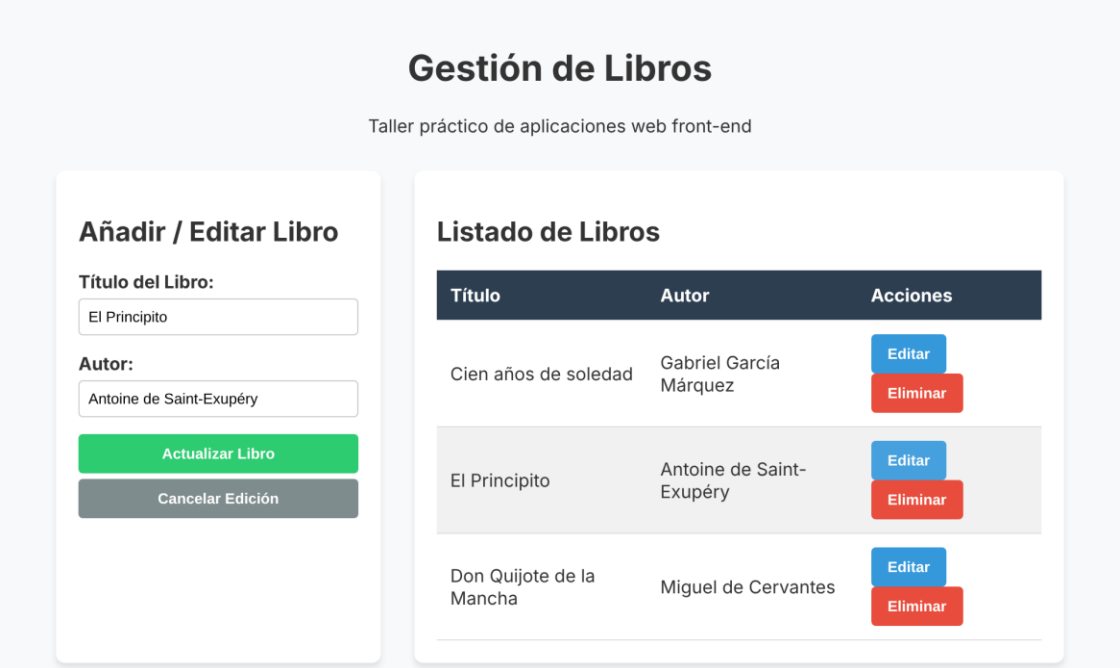


Figura 4. **Actualizar (preparación)**: al pulsar “Editar” los datos pasan al formulario, el botón cambia a “Actualizar Libro” y aparece “Cancelar Edición”.

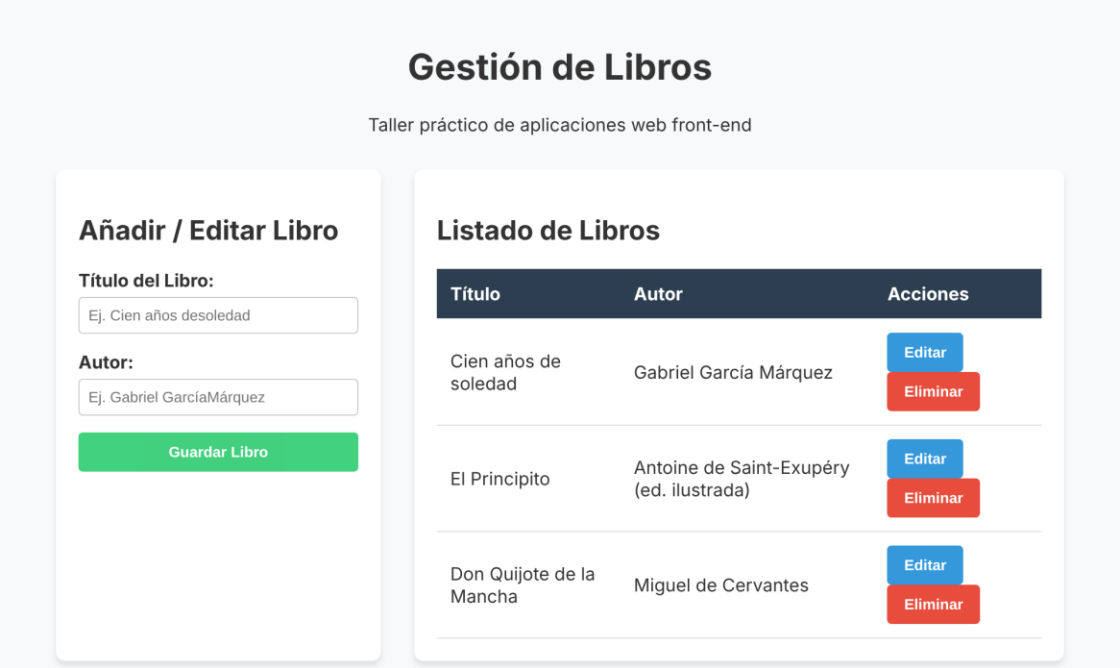


Figura 5. **Actualizar**: el autor de “El Principito” fue modificado y la tabla se refrescó.

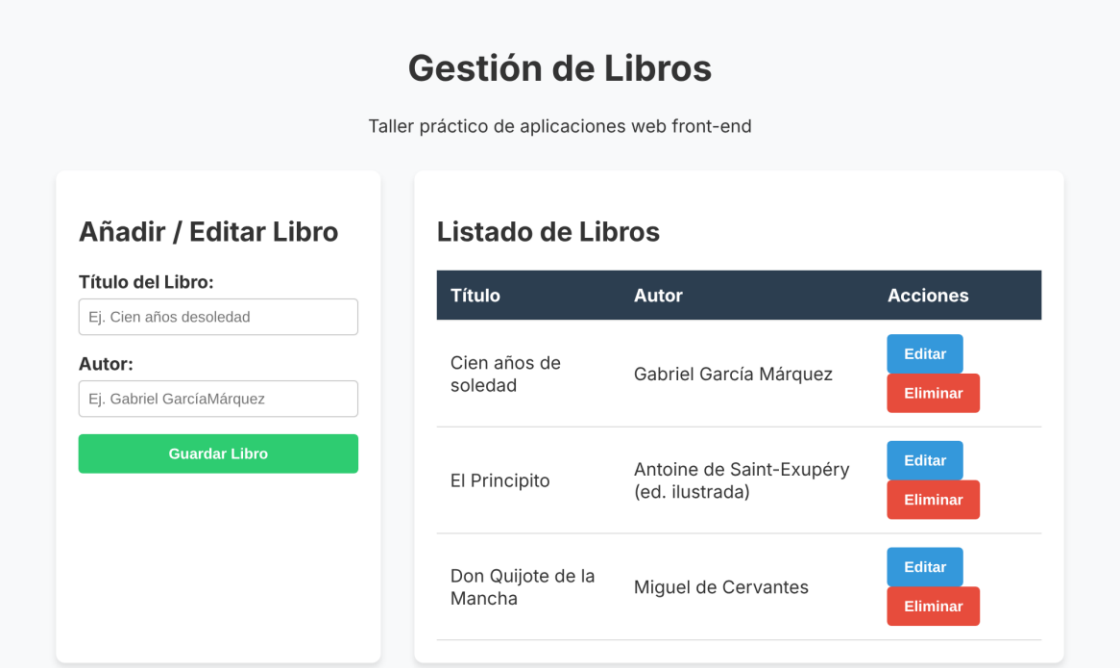


Figura 6. **Persistencia**: tras recargar la página (F5) los datos se conservan gracias a `localStorage`.

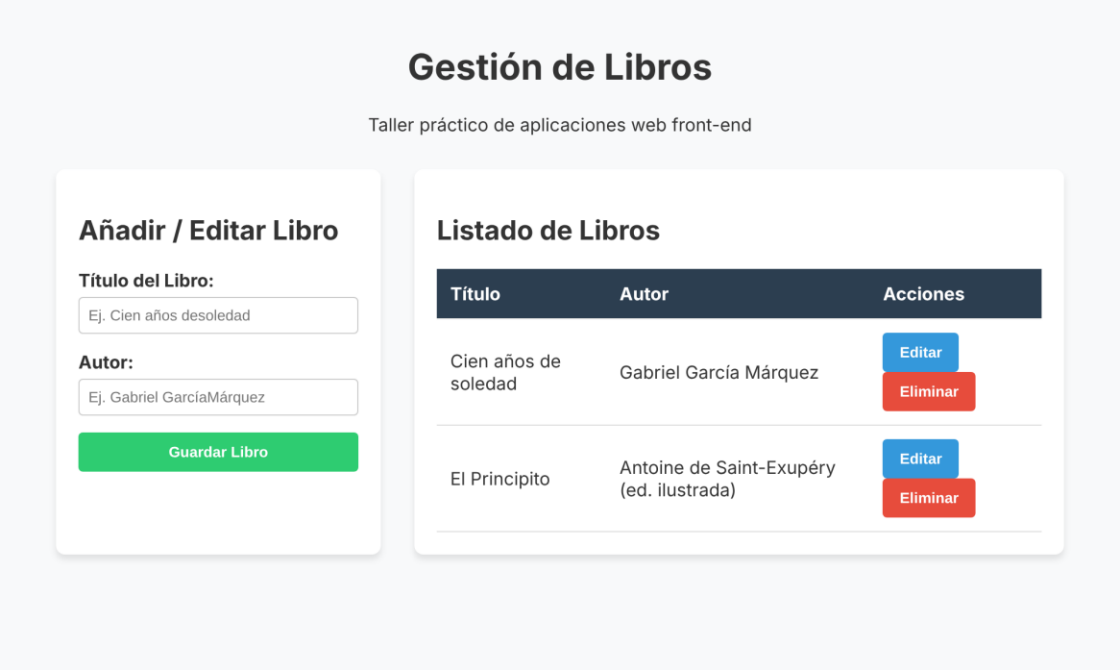


Figura 7. **Eliminar**: tras confirmar el cuadro de diálogo, el libro "Don Quijote de la Mancha" fue quitado de la lista.

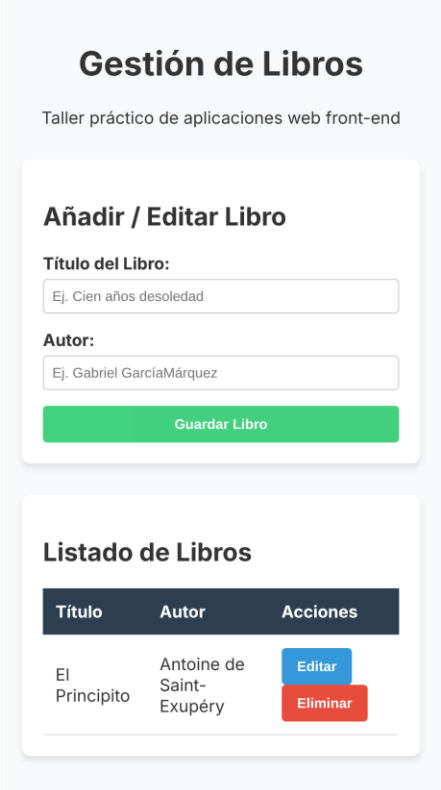


Figura 8. Diseño responsivo en pantalla móvil (420 px): el formulario y la tabla se apilan en una sola columna.

11. Resumen de pruebas realizadas

Prueba	Resultado esperado	Resultado
Agregar un libro	Aparece una nueva fila en la tabla	Correcto
Editar un libro	Los datos se cargan en el formulario y se actualizan al guardar	Correcto

Prueba	Resultado esperado	Resultado
Eliminar un libro	Se pide confirmación y se quita la fila	Correcto
Recargar la página	Los libros se conservan	Correcto
Campos vacíos	El navegador impide enviar el formulario (`required`)	Correcto
Consola del navegador	Sin errores de JavaScript	Correcto

12. Observaciones y posibles mejoras

- **Persistencia al borrar el último libro**: en `renderizarLibros()`, el `return` del caso “lista vacía” ocurre antes de `localStorage.setItem`. Por ello, al eliminar el último libro la tabla queda vacía, pero al recargar la página reaparece. Se comprobó en la ejecución. Solución: mover la línea de `localStorage.setItem` antes del `if`.
- **Seguridad**: los datos se insertan con `innerHTML`; si alguien escribe código HTML en el título podría ejecutarse. Una mejora es usar `textContent` o escapar los caracteres especiales.
- **Comentario partido**: en `eliminarLibro`, la palabra “formulario” quedó en una línea aparte del comentario. No genera error, pero conviene unirla al comentario.